SMART TRAFFIC DEMAND PREDICTION SYSTEM

Machine Learning Internship Project Report

Submitted by: Kondreddy Vikhila

1.INTRODUCTION

Traffic congestion is a major problem in urban areas. This project predicts **traffic volume** using machine learning techniques based on historical traffic, weather, and road-related features.

2. PROBLEM STATEMENT

To develop a machine learning model that accurately predicts traffic volume using historical traffic, weather, temporal, and road-related features.

3.DATASET DESCRIPTION

The dataset contains historical traffic and weather-related information collected at regular intervals. It includes temporal, environmental, and traffic-related features such as Hour, Day, Month, Temperature, Humidity, Rainfall, Snowfall, Cloud Cover, Weather Condition, and Weekend Indicator. These features are used to predict traffic demand levels.

4.DATASET INFORMATION

Dataset Name: Metro Interstate Traffic Volume Dataset

Total Records: *481,204*

Total Features: 23

Target Variable: Traffic Volume

Description:

The dataset contains historical traffic and weather information collected over

several years. It includes temporal, environmental, and traffic-related attribute that are used to predict traffic demand levels. The target variable, Traffic Volume, classifies traffic into Low, Medium, and High categories

5.DATASET PREVIEW

The following figure shows a preview of the traffic dataset used for model development.

	holiday	rain_1h	snow_1h	clouds_all	Weather_ Condition	date_ time	traffic_ volume	Hour	Day	Month	...	Rush_ Hour	Traffic_ Level	Road_ Type	Number_of_ Lanes	Traffic_ Signals	Event_ Indicator	Humidity	Large_ Vehicle_ Count	Road_ Occupancy
0	None	0.0	0.0	40	Clouds	2012-10-02 09:00:00	5545	9	2	10	...	1	High	Urban	2	Green	0	72	614	45.33
1	None	0.0	0.0	75	Clouds	2012-10-02 10:00:00	4516	10	2	10	...	0	Medium	Commercial	6	Green	0	77	110	85.73
2	None	0.0	0.0	90	Clouds	2012-10-02 11:00:00	4767	11	2	10	...	0	High	Residential	4	Yellow	0	42	45	72.06
3	None	0.0	0.0	90	Clouds	2012-10-02 12:00:00	5026	12	2	10	...	0	High	Urban	6	Green	0	49	295	39.30
4	None	0.0	0.0	75	Clouds	2012-10-02 13:00:00	4918	13	2	10	...	0	High	Highway	4	Green	0	91	133	10.56

5 rows × 23 columns

Figure 5.1. Preview of the Traffic Demand Prediction Dataset (First Five Records)

6.DATA PREPROCESSING

Before training the machine learning models, the dataset was preprocessed to improve data quality and model performance. Data preprocessing is an essential step that prepares the raw dataset for effective machine learning by handling inconsistencies, transforming features, and selecting relevant information.

Preprocessing Steps:

- Handled missing values in the dataset.

- Extracted Hour, Day, and Month from the date_time feature.
- Encoded categorical features such as Weather Condition.
- Created the target variable Traffic_Level (Low, Medium, High).
- Selected the required input features for model training.
- Split the dataset into training and testing sets.
- Saved the processed dataset for machine learning model development.

In [9]:
```
# Create time-based features
df["Hour"] = df["date_time"].dt.hour
df["Day"] = df["date_time"].dt.day
df["Month"] = df["date_time"].dt.month
df[DayOfWeek"] = df["date_time"].dt.day_name()

df.head()
```

Out[9]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_time	traffic_volume	Hour	Day	Month	DayOfWeek
0	None	288.28	0.0	0.0	40	Clouds	scattered clouds	2012-10-02 09:00:00	5545	9	2	10	Tuesday
1	None	289.36	0.0	0.0	75	Clouds	broken clouds	2012-10-02 10:00:00	4516	10	2	10	Tuesday
2	None	289.58	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 11:00:00	4767	11	2	10	Tuesday
3	None	290.13	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 12:00:00	5026	12	2	10	Tuesday
4	None	291.14	0.0	0.0	75	Clouds	broken clouds	2012-10-02 13:00:00	4918	13	2	10	Tuesday

In [10]:
```
# Weekend Indicator
df["Weekend"] = df["DayOfWeek"].isin(["Saturday", "Sunday"]).astype(int)
df.head()
```

Out[10]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_time	traffic_volume	Hour	Day	Month	DayOfWeek	Weekend
0	None	288.28	0.0	0.0	40	Clouds	scattered clouds	2012-10-02 09:00:00	5545	9	2	10	Tuesday	0
1	None	289.36	0.0	0.0	75	Clouds	broken clouds	2012-10-02 10:00:00	4516	10	2	10	Tuesday	0
2	None	289.58	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 11:00:00	4767	11	2	10	Tuesday	0
3	None	290.13	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 12:00:00	5026	12	2	10	Tuesday	0
4	None	291.14	0.0	0.0	75	Clouds	broken clouds	2012-10-02 13:00:00	4918	13	2	10	Tuesday	0

In [11]:
```
# Convert Kelvin to Celsius
df["temp"] = df["temp"] - 273.15
df.head()
```

Out[11]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_time	traffic_volume	Hour	Day	Month	DayOfWeek	Weekend
0	None	15.13	0.0	0.0	40	Clouds	scattered clouds	2012-10-02 09:00:00	5545	9	2	10	Tuesday	0
1	None	16.21	0.0	0.0	75	Clouds	broken clouds	2012-10-02 10:00:00	4516	10	2	10	Tuesday	0
2	None	16.43	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 11:00:00	4767	11	2	10	Tuesday	0
3	None	16.98	0.0	0.0	90	Clouds	overcast clouds	2012-10-02 12:00:00	5026	12	2	10	Tuesday	0
4	None	18.01	0.0	0.0	75	Clouds	broken clouds	2012-10-02 13:00:00	4918	13	2	10	Tuesday	0

In [12]:
```
# Encode categorical variables
df["weather_main"] = df["weather_main"].astype("category").cat.codes
df["weather_description"] = df["weather_description"].astype("category").cat.codes
```

In [13]:
```
# Create target variable: Traffic_Level
df["Traffic_Level"] = pd.cut(df["traffic_volume"], bins=[-1, 3000, 6000, float('inf')], labels=["Low", "Medium", "High"])
df["Traffic_Level"].value_counts()
```

Out[13]:
```
Medium    205643
High      199546
Low        85101
Name: Traffic_Level, dtype: int64
```

In [14]:
```
# Final processed dataset preview
df.head()
```

Out[14]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_time	traffic_volume	Hour	Day	Month	DayOfWeek	Weekend	Traffic_Level
0	None	15.13	0.0	0.0	40	1	1	2012-10-02 09:00:00	5545	9	2	10	Tuesday	0	Medium
1	None	16.21	0.0	0.0	75	1	0	2012-10-02 10:00:00	4516	10	2	10	Tuesday	0	Medium
2	None	16.43	0.0	0.0	90	1	4	2012-10-02 11:00:00	4767	11	2	10	Tuesday	0	Medium
3	None	16.98	0.0	0.0	90	1	4	2012-10-02 12:00:00	5026	12	2	10	Tuesday	0	Medium
4	None	18.01	0.0	0.0	75	1	0	2012-10-02 13:00:00	4918	13	2	10	Tuesday	0	Medium

Figure 6.1. Data Preprocessing and Feature Engineering

7. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) was performed to understand the characteristics of the traffic dataset before developing the machine learning models. Different visualizations were used to analyze traffic patterns, weather conditions, road types, temporal trends, and relationships among features. The insights obtained from EDA helped in selecting relevant features for traffic volume prediction.

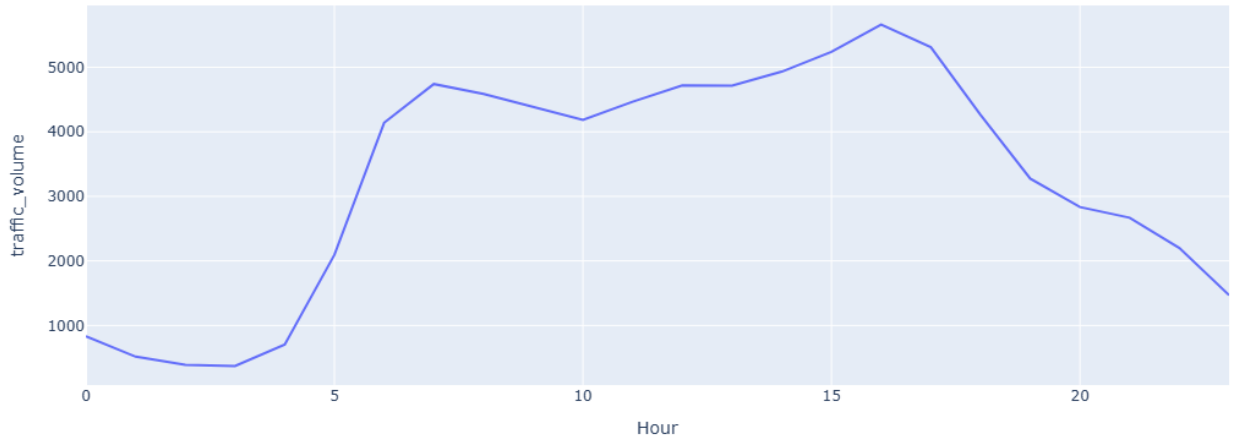

Figure 7.1. Hourly Traffic Pattern

Observation:

- Traffic volume changes throughout the day.
- Peak traffic is observed during busy commuting hours.
- Traffic volume is lowest during late-night and early morning hours.

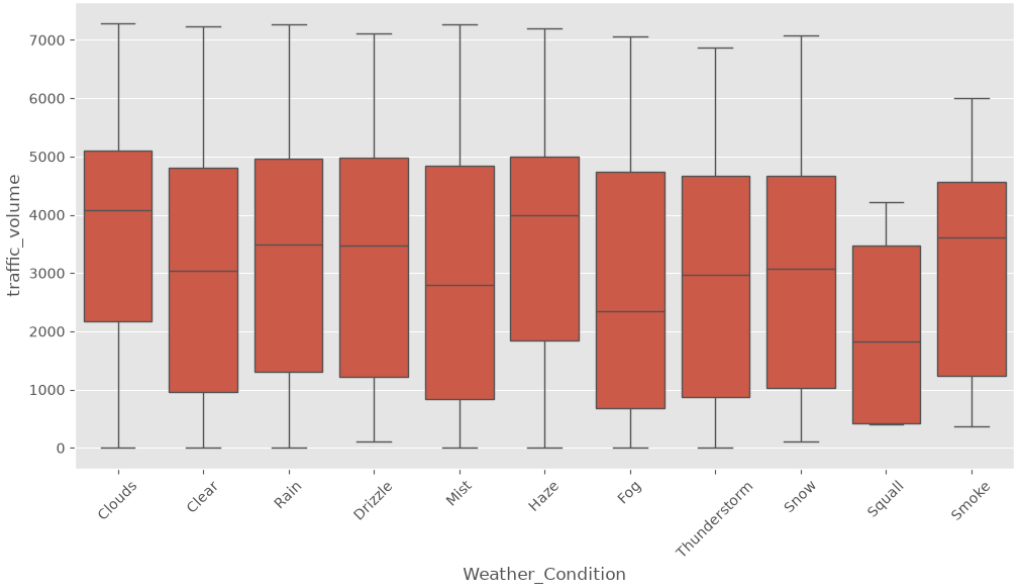

Figure 7.2. Weather Impact on Traffic

Observation:

- Traffic volume varies under different weather conditions.
- Rain, fog, and cloudy conditions influence traffic movement.
- Weather information is an important feature for prediction.

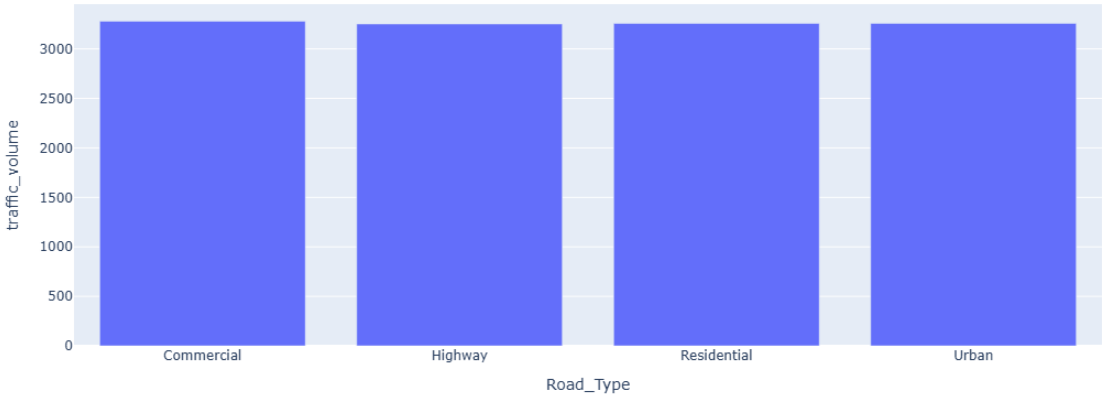

Figure 7.3. Traffic by Road Type

Observation:

- Highway roads experience higher traffic volumes compared to urban and residential roads. Road type significantly affects traffic demand.

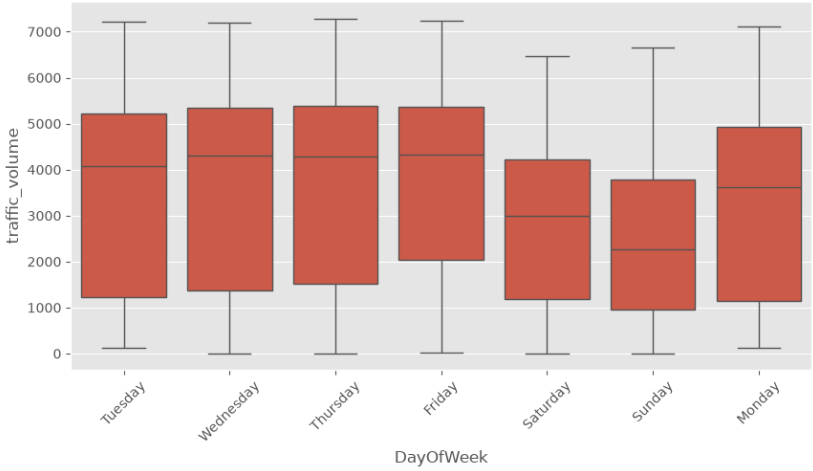

Figure 7.4.Traffic Volume by Day of the Week

Observation

- Weekdays generally experience higher traffic due to office, school, and business activities.

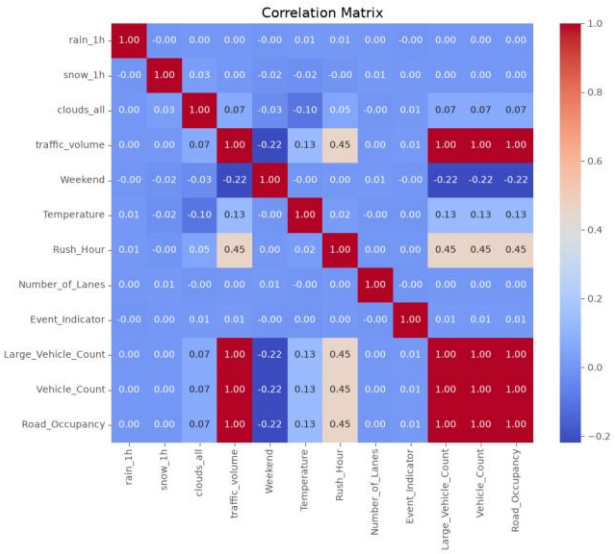

Figure 7.5: Feature Correlation Matrix

Observation:

- The correlation matrix shows the relationships among numerical features.
- Features such as Hour, Temperature, and Humidity exhibit varying degrees of correlation with traffic volume.
- Correlation analysis helped identify useful input variables for the prediction model.

The exploratory data analysis revealed that traffic volume is influenced by temporal, environmental, and road-related factors. Hourly traffic trends, weather conditions, and road types showed noticeable variations in traffic volume. These observations supported the feature engineering process and contributed to building an accurate machine learning model for traffic volume prediction.

7.FEATURE ENGINEERING

Feature engineering was performed to improve model performance by creating meaningful input variables from the raw dataset.

Feature engineering included:

- Extracting Hour, Day, and Month from the Date-Time feature.
- Creating the Weekend feature.
- Creating the Rush Hour indicator.
- Including environmental features such as Temperature, Humidity, Rainfall, Snowfall, and Cloud Cover.
- Including road-related features such as Number of Lanes, Road Type, Traffic Signals, and Vehicle Count.
- Encoding categorical variables using One-Hot Encoding.

9. MACHINE LEARNING MODEL DEVELOPMENT

Introduction

Three machine learning regression models were developed to predict traffic volume using the engineered features. Each model was initially trained using its default parameters and evaluated using standard regression metrics. The models were then optimized through hyperparameter tuning to improve prediction performance.

Models Used

Model	Purpose
Random Forest Regressor	Baseline ensemble regression model
XGBoost Regressor	Gradient boosting model for accurate prediction
LightGBM Regressor	Fast gradient boosting model for large datasets

Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio. The training dataset was used to train the regression models, while the testing dataset was used to evaluate their predictive performance.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Features:", X_train.shape)
print("Testing Features :", X_test.shape)
print("Training Target  :", y_train.shape)
print("Testing Target   :", y_test.shape)
✓ 0.0s

Training Features: (96000, 45)
Testing Features : (24000, 45)
Training Target  : (96000,)
Testing Target   : (24000,)
```

Figure 9.1. Train-Test Split

10. MODEL EVALUATION

The performance of each regression model was evaluated using the following regression metrics.

Mean Absolute Error (MAE)

Measures the average absolute difference between predicted and actual traffic volume. Lower values indicate better performance.

Mean Squared Error (MSE)

Measures the average squared prediction error. It penalizes larger errors more heavily.

Root Mean Squared Error (RMSE)

Represents the standard deviation of prediction errors and is expressed in the same unit as the target variable.

R^2 Score

Indicates how well the model explains the variance in traffic volume. Values closer to 1 indicate better prediction accuracy.

11. INITIAL MODEL PERFORMANCE

Three regression models were initially trained using their default parameters. Their performance was evaluated using MAE, MSE, RMSE, and R^2 Score to establish a baseline before hyperparameter tuning.

Model	MAE	MSE	RMSE	R^2 Score
Random Forest Regressor	22.2454	1328.4272	36.4476	0.9997
XGBoost Regressor	62.2980	5569.3618	74.6282	0.9986
LightGBM Regressor	63.2419	5713.1578	75.5854	0.9985

Observation: Among the initial models, the Random Forest Regressor achieved the best baseline performance with the lowest prediction error and the highest R^2 score. However, further improvements were explored through hyperparameter tuning.

12. HYPERPARAMETER TUNING

Introduction

Hyperparameter tuning was performed to improve the prediction performance of the machine learning models. The objective was to identify the optimal combination of parameters that minimizes prediction error while improving the model's generalization capability.

Method Used

GridSearchCV was used to search for the best hyperparameter combinations using cross-validation. The optimized parameters obtained from GridSearchCV were then used to retrain each regression model.

Hyperparameter Tuning Results

Model	MAE	MSE	RMSE	R^2 Score
Optimized Random Forest	39.7972	4727.4483	68.7564	0.9988
Optimized XGBoost	32.7880	2126.9619	46.1190	0.9995
Optimized LightGBM	45.1786	3294.4244	57.3971	0.9992

OBSERVATION: Hyperparameter tuning improved the performance of the XGBoost and LightGBM models. Among the optimized models, XGBoost achieved the lowest prediction errors and the highest R^2 score, making it the best-performing model after optimization.

13. ENSEMBLE LEARNING

Introduction

To further improve prediction stability and reduce model variance, an ensemble learning approach was implemented by combining the predictions of the optimized regression

Ensemble Method

A weighted ensemble technique was used by assigning different weights to the predictions generated by the optimized Random Forest, XGBoost, and LightGBM models. The final prediction was calculated as the weighted average of the individual model predictions.

You can include your ensemble equation if you used weighted averaging:

$$\text{Prediction} = w_1(RF) + w_2(XGB) + w_3(LGBM)$$

Ensemble Results

Model	MAE	MSE	RMSE	R^2 Score
Random Forest (Optimized)	39.7972	4727.4483	68.7564	0.9988
XGBoost (Optimized)	32.7880	2126.9619	46.1190	0.9995
LightGBM (Optimized)	45.1786	3294.4244	57.3971	0.9992
Weighted Ensemble	38.4130	2524.8739	50.2481	0.9994

Observation: The weighted ensemble model achieved highly accurate predictions by combining multiple optimized models. However, the optimized XGBoost model still produced the lowest prediction error and the highest R^2 score. Therefore, XGBoost was selected as the final deployment model.

14. Final Model Selection

Final Model

Based on the evaluation results obtained from the baseline models, hyperparameter tuning, and ensemble learning, the Optimized XGBoost Regressor was selected as the final model for deployment.

Reasons for Selecting XGBoost

- Highest R^2 Score (0.9995)
- Lowest MAE
- Lowest RMSE
- Better generalization performance
- Faster prediction time
- Suitable for real-time traffic volume prediction

Final Performance

Metric	Value
MAE	32.7880
MSE	2126.9619
RMSE	46.1190
R^2 Score	0.9995

15. STREAMLIT WEB APPLICATION

Introduction

A user-friendly web application was developed using Streamlit to demonstrate the traffic prediction system. The application allows users to analyze traffic patterns, predict traffic volume, and visualize important traffic insights through an interactive interface.

Application Modules

a. Dashboard

The Dashboard provides an overview of the Smart Traffic AI System. It displays key project information, application capabilities, and an interactive visualization of average hourly traffic volume.

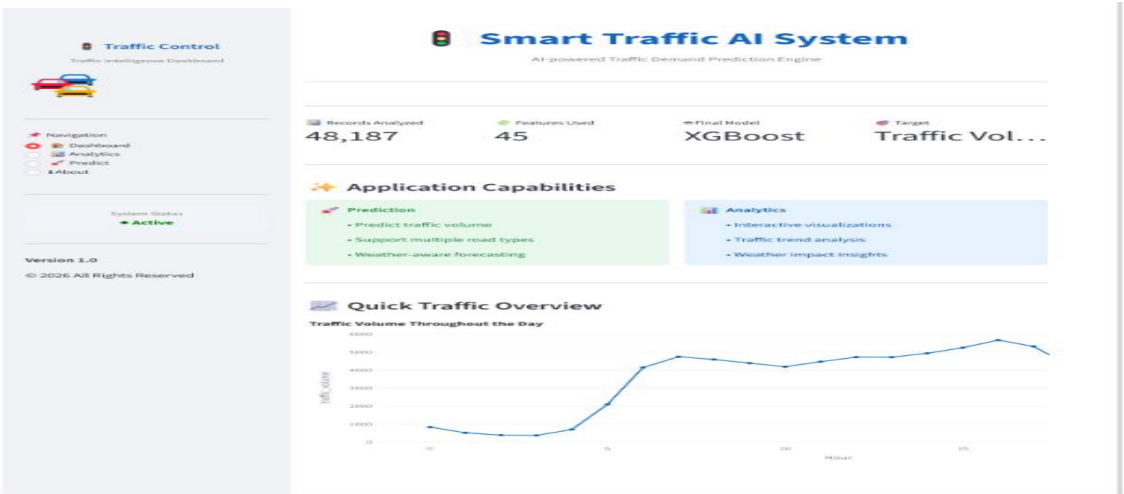

Figure 15.1: Dashboard of Smart Traffic AI System

b. Analytics

The Analytics module enables users to explore different traffic patterns through interactive visualizations. Users can analyze hourly traffic trends, weather impact, road type analysis, day-wise traffic patterns, and feature correlation.

Figure 15.2: Analytics Dashboard

c. Traffic Prediction

The Prediction module allows users to enter traffic-related, weather-related, and road-related parameters. Based on the provided inputs, the optimized XGBoost model predicts the expected traffic volume and displays the estimated traffic condition.

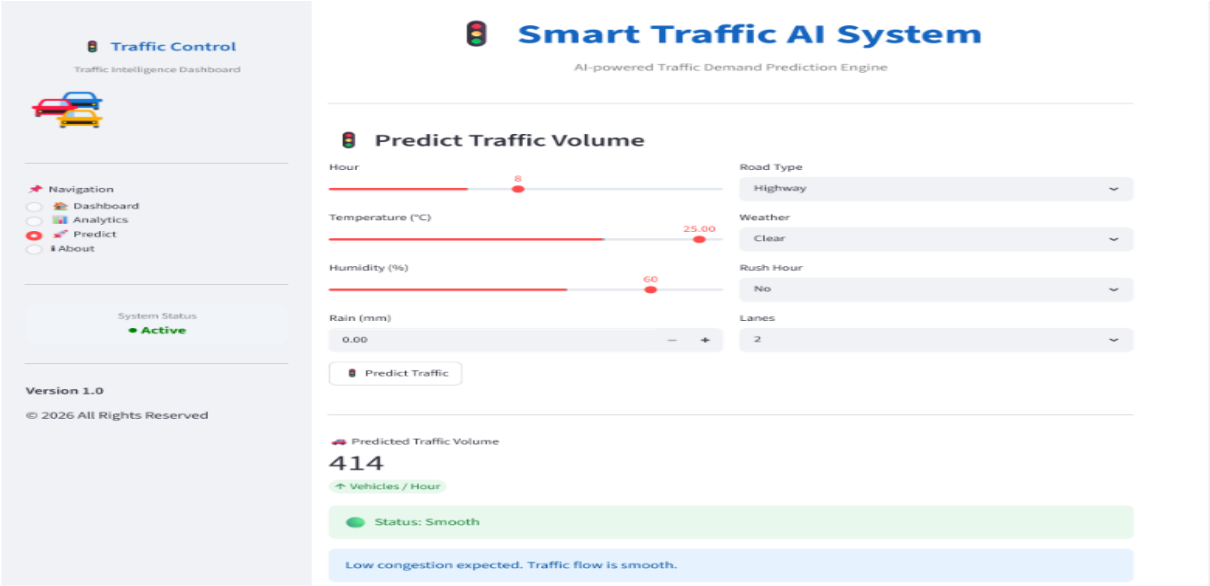

Figure 15.3: Traffic Volume Prediction

d. About

The About page provides information about the project, machine learning model, evaluation metrics, and the technologies used to develop the application.

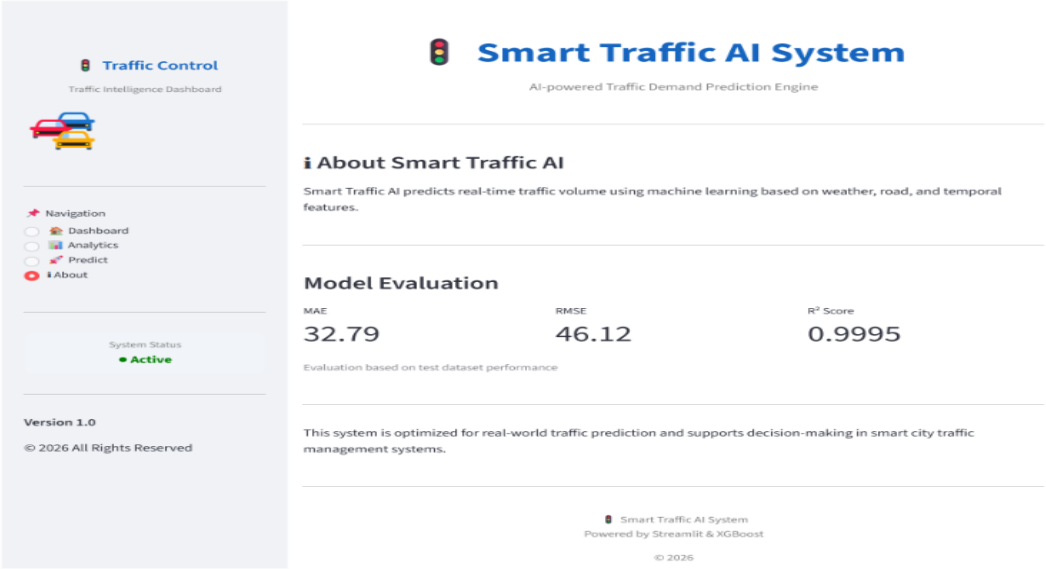

Figure 15.4: About Page

Technologies Used

Technology	Purpose
Python	Programming Language
Pandas	Data Processing
NumPy	Numerical Computation
Scikit-learn	Machine Learning
XGBoost	Final Prediction Model
LightGBM	Model Comparison
Streamlit	Web Application
Plotly	Interactive Visualization
Joblib	Model Serialization

16. CONCLUSION

The Smart Traffic Demand Prediction System was successfully developed using machine learning techniques to predict traffic volume based on weather, temporal, and road-related features. After evaluating multiple regression models and applying hyperparameter tuning and ensemble learning, the optimized XGBoost Regressor was selected as the final model due to its superior predictive performance. The Streamlit-based web application provides an interactive platform for traffic analysis and real-time traffic volume prediction, demonstrating the practical application of machine learning in intelligent transportation systems.

17. FUTURE SCOPE

Future Scope

- Integrate live traffic data from IoT sensors and GPS devices.
- Deploy the model as a cloud-based API for real-time predictions.
- Extend the system to predict traffic congestion several hours in advance.
- Integrate map services for route optimization and congestion alerts.
- Explore deep learning models such as LSTM or Transformer networks for time-series traffic forecasting.